

P1779R1: ABI isolation for member functions

Audience: EWG
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
July 31, 2019

History

- r1:
- Added explicit instantiation case
 - Discussed closure types and defaulted member functions
 - Discussed timing of any performance loss
 - Added missing editing instruction

Background

Traditionally, a library’s functions that are not inline (and are not function templates) must be declared and defined separately because the definition must appear only once and the declaration must appear in every translation unit that uses it. This approach has the obvious disadvantage of redundancy and verbosity—especially in the (somewhat uncommon) cases of member functions of nested classes and explicitly instantiated function templates—and associated opportunity for error, whether due to different contexts for the two declarations or their divergence over time. One of the benefits of modules is that only a definition is required for each function.

[P1498R1](#) gives the `inline` keyword back something of its original meaning, in that the prohibition on using internal-linkage names from an inline function is meant to support an implementation strategy of inlining only inline functions so as to avoid emitting undefined symbols that must be resolved against “internal-linkage” entities. The practical effect is that internal linkage symbols are never part of the ABI of a module, but also that symbols with module linkage may or may not be depending on where they are used. The `inline` keyword therefore allows a module author to choose, for each exposed function, whether to expose its implementation to importers, gaining the possibility of better performance at the expense of tighter coupling (and thus stronger restrictions on evolution of the library code and/or more frequent recompiles for the client).

Member functions and friend functions defined in a class definition are implicitly inline for the practical reason that (per the ODR) they appear in every translation unit that `#includes` the (single) class definition. Actually inlining them may or may not be significant; the recompilation cost of doing so is paid anyway with typical modification-time-based build systems. Their status does, however, cause `[temp.explicit]/13` to apply, disabling explicit instantiation declarations for them. Since explicit instantiation *definitions* in the interface of a module have the effect of declarations in an importing translation unit, a member function template defined within its class cannot use explicit instantiation to guard access to internal-linkage names.

Proposal

This paper proposes removing the implicit inline status from functions defined in a class definition attached to a (named) module. This allows classes to benefit from avoiding redundant declarations, maintaining the flexibility offered to module authors in declaring functions with or without `inline`.

This proposal does not change the fact that the function call operator and conversion function (if any) of a closure type are inline (`[expr.prim.lambda.closure]/3, /11`). Closure types are therefore affected by the restrictions of P1498R1, which usefully prevents ABI problems based on their invented names.

Neither does it change that implicitly-declared class members are inline (`[class.default.ctor]`, `[class.copy.ctor]`, `[class.copy.assign]`, `[class.dtor]`), as is a function explicitly defaulted on its first declaration if it can be (`[dcl.fct.def.default]`). Such members merely invoke member functions of the types of the data members; the functions are usable if the types may be used for the members.

Tony table

In the following, the functions `helper` are kept out of the ABI of the library presented.

C++17	N4820	
a.hpp:	a.mpp:	a.mpp:
<pre>#ifndef A_HPP #define A_HPP void output(int,float,const char *what); template<class T> void label(T,const char *what); extern template void label(int,const char*); extern template void label(float,const char*); struct A { A(int,float); void output(const char *what) const; template<int> void scale(const char *what) const; private: float x; }; extern template void A::scale<1>(const char*) const; extern template void A::scale<10>(const char*) const; extern template void A::scale<100>(const char*) const; #endif</pre>	<pre>export module a; import<iostream>; import<string>; namespace { float helper(int i,float f) {return f/i+i;} std::string_view helper(const char *what) {return std::string_view(what).substr(0,8);} } void output(int i,float f,const char *what) { std::cout << helper(what) << ": " << helper(i,f) << '\n'; } template<class T> void label(T t,const char *what) { std::cout << helper(what) << ": " << t << '\n'; } template void label(int,const char*); template void label(float,const char*);</pre>	<pre>export import< import< namespace float std:: {retu } void ou std:: } templat std:: } templat templat</pre>

a.cpp:

```
#include<iostream>
#include<string>
#include"a.hpp"

namespace {
    float helper(int i,float f) {return f/i+i;}
    std::string_view helper(const char *what)
    {return std::string_view(what).substr(0,8);}
}

void output(int i,float f,const char *what) {
    std::cout << helper(what) << ": " << helper(i,f) << '\n';
}

template<class T> void label(T t,const char *what) {
    std::cout << helper(what) << ": " << t << '\n';
}

template void label(int,const char*);
template void label(float,const char*);

A::A(int i,float f) : x(helper(i,f)) {}

void A::output(const char *what) const {
    std::cout << helper(what) << ": " << x << '\n';
}

template<int I> void A::scale(const char *what) const {
    std::cout << helper(what) << ": " << x*I << '/' << I << '\n';
}

template void A::scale<1>(const char*) const;
template void A::scale<10>(const char*) const;
template void A::scale<100>(const char*) const;
```

```
struct A {
    A(int,float);
    void output(const char *what) const;
    template<int> void scale(const char *what) const;
private:
    float x;
};

A::A(int i,float f) : x(helper(i,f)) {}

void A::output(const char *what) const {
    std::cout << helper(what) << ": " << x << '\n';
}

template<int I> void A::scale(const char *what) const {
    std::cout << helper(what) << ": " << x*I << '/' << I << '\n';
}

template void A::scale<1>(const char*) const;
template void A::scale<10>(const char*) const;
template void A::scale<100>(const char*) const;
```

```
struct A {
    A(int
    void
    std
    }
    templ
    std
    }
private
    float
};
templat
templat
```

The change between the middle and right columns is not vast, but it provides consistency between member and non-member functions. It avoids the need to define every member function of a class template out of line (each with its own *template-head*) in order to use internal-linkage helpers via an explicit instantiation of the class. It also allows making helpers in the common namespace `detail` approach truly private merely by removing the namespace name everywhere without changing any class definitions.

Concerns

While a mechanical conversion of a header containing such implicitly inline functions into a module may produce a performance degradation with this proposal, it is trivial to add `inline` to whatever appropriate set of member/friend function definitions, and doing so does not affect its meaning in C++17. On the other hand, this proposal may ease the conversion of certain headers that (in a violation of the ODR that normally goes unnoticed and vanishes upon modulation) use helper functions with internal linkage in such functions. It would also be reasonable for implementations to provide a (conforming) mode that inlines functions in modules regardless of the keyword; some applications may wish to use it regularly (at the expense of additional recompilation), and others may use it temporarily while adopting modules.

It is also conceivable that this proposal will have an indirect, adverse impact on compile performance because it encourages adding code to module interface units whose modification triggers recompilation. The popularity of header-only libraries suggests that this is often considered unimportant in practice. Obviously, any users concerned can continue to define member/friend functions in module implementation units.

Evolution has rejected a [previous proposal](#) to “clean up” language rules in a module context, for fear of creating a distinct “modules dialect” within the language. This proposal does not tend to create such a dialect because it does not change the meaning of or add restrictions to modular code: it merely suppresses a rule that exists for practical reasons in a context where those reasons do not pertain and where the rule would already have new and potentially undesirable substantive effects.

Since the effect of this proposal is largely restricted to removing the P1498 restrictions on functions that are no longer inline, it could perhaps be considered after C++20. Any performance regression would, however, then arise from a change of language version *alone* rather than from inattention when already making other code changes to use modules. It is also simpler, and avoids any compatibility concerns, to make the change beforehand.

Wording

Relative to N4820.

Remove [dcl.inline]/4:

- A function defined within a class definition is an inline function.

Change [class.mfct]/1:

- A member function may be defined ([dcl.fct.def]) in its class definition, in which case it is an inline member function ([dcl.inline]) **if it is attached to the global module**, or it may be defined outside of its class definition if it has already been declared but not defined in its class definition. A member function definition that appears outside of the class definition shall appear in a namespace scope enclosing the class definition. Except for member function definitions that appear outside of a class definition, and except for explicit specializations of member functions of class templates and member function templates ([temp.spec]) appearing outside of the class definition, a member function shall not be redeclared.

Change [class.friend]/7:

- Such a function is implicitly an inline function ([dcl.inline]) **if it is attached to the global module**. A friend function defined in a class is in the (lexical) scope of the class in which it is defined. A friend function defined outside the class is not ([basic.lookup.unqual]).

Acknowledgments

Thanks to Richard Smith for pointing out the different kinds of performance regression.